# Institutional Quantitative Research Infrastructure

## M/G/$\infty$ Queue-Based Valuation Mean-Reversion Portfolio System

A Complete Step-by-Step Guide to Building a
Fixed-Universe Quantitative Database
and Walk-Forward Optimization Engine
for $0

Covering: Fixed-Universe Construction (Russell 3000, 2025) · Point-in-Time Fundamentals
PostgreSQL Schema Design · Python Data Pipeline
M/G/$\infty$ Parameter Extraction · Walk-Forward Backtesting

*Note: Survivorship bias is intentionally not corrected in this version.*
*The universe is fixed to the 2025 Russell 3000 constituent list.*
*This is a mathematical research prototype, not a production trading system.*

Quantitative Research Division

Version 1.0 · 2025

This document is intended for educational and research purposes only.
Nothing herein constitutes investment advice.

# Preface

This document is a complete technical blueprint for constructing an institutional-grade quantitative trading research environment from scratch, at zero cost. It bridges theoretical operations research—specifically, Erlang's M/G/$\infty$ queuing model—with the practical engineering required to deploy it against real market data.

**The Core Thesis.** Portfolio capital allocation is mathematically equivalent to a service queue. Buy signals arrive randomly at some rate $\lambda_i$ per sector $i$; capital is deployed for a random holding duration with mean $\mu_i^{-1}$. The M/G/$\infty$ model provides a closed-form, provably optimal formula for how much capital to reserve in each sector bucket to keep stockout risk (running out of cash when signals arrive) equal across all sectors. The Square-Root Rule is the practical output of this model.

The pipeline described here spans four phases:

**Phase 1.** **Universe Construction** — Download the current (2025) Russell 3000 constituent list from the iShares IWV ETF holdings file. This is a single CSV download. All tickers are fixed for the entire backtest.

**Phase 2.** **Database Architecture** — Design a PostgreSQL schema that is Point-in-Time (PiT) correct for fundamental data, with a fixed static universe.

**Phase 3.** **Data Ingestion** — Populate the database using the SEC EDGAR API for XBRL financial data and `yfinance` for prices, fetching from 2010 to present for every ticker in the fixed universe.

**Phase 4.** **Strategy & Validation** — Extract M/G/$\infty$ parameters ($\lambda$, $\mu$) from the database and run Walk-Forward Optimization from 2010 to 2025.

**Survivorship Bias Acknowledgement.** This version of the pipeline uses the *current* Russell 3000 (as of 2025) as a fixed universe. Companies that were delisted, went bankrupt, or were acquired between 2010 and 2025 are **excluded**. This means the backtest will produce inflated returns relative to what a real strategy would have achieved—value traps and bankruptcies are missing from the signal set. This is a deliberate simplification: the goal of this document is to build and validate the M/G/$\infty$ allocation mathematics. Survivorship bias correction is left as a future extension once the core engine is verified.

# Contents

**Preface** **1**

**I Theoretical Foundation** **4**

**1 The M/G/∞ Queue and the Square-Root Rule** **5**
1.1 Why Queuing Theory Belongs in Finance . . . 5
1.2 The M/G/∞ Model . . . 5
1.2.1 Model Specification . . . 5
1.2.2 Steady-State Distribution . . . 5
1.2.3 The Capital Buffer Problem . . . 6
1.3 The Square-Root Allocation Rule . . . 6
1.3.1 Derivation . . . 6
1.3.2 Sector Budgeting . . . 6
1.3.3 Numerical Example . . . 7
1.4 Why Valuation-Based Signals, Not Price . . . 7

**II Data Infrastructure** **8**

**2 Universe Construction: Fixed Russell 3000 (2025)** **9**
2.1 Design Decision: Fixed Universe . . . 9
2.2 Parsing `IWV_holdings25.csv` . . . 9
2.3 Fundamental Data Strategy: yfinance vs. SEC EDGAR . . . 10

**3 PostgreSQL Database Schema** **12**
3.1 Architecture Overview . . . 12
3.2 Table Definitions . . . 12
3.2.1 Table 1: `tickers_metadata` — The Master Roster . . . 12
3.2.2 Table 2: `daily_prices` — Price History . . . 13
3.2.3 Table 3: `fundamentals` — Quarterly Financials . . . 13

**4 Data Ingestion Pipeline** **15**
4.1 Phase 1: Universe Loading from IWV CSV . . . 15
4.2 Phase 2: Price Data via yfinance . . . 16
4.3 Phase 3: Fundamental Data via yfinance . . . 18

**III Strategy Implementation** **21**

**5 Extracting M/G/∞ Parameters from the Database** **22**
5.1 The Valuation Signal . . . 22
5.1.1 Computing P/E Ratios at a Simulation Date . . . 22

5.1.2 Generating Signals: The 2-SD Threshold . . . . . 23
5.2 Computing $\lambda_i$ and $\mu_i$ from Historical Data . . . . . 24

**6 Walk-Forward Optimization** **28**
6.1 Methodology . . . . . 28
6.1.1 Window Configuration . . . . . 28
6.2 WFO Engine Implementation . . . . . 29
6.3 Interpreting Walk-Forward Results . . . . . 32

**7 Regime Analysis and Stress Testing** **34**
7.1 Key Regimes in the WFO Window (2022–2025) . . . . . 34
7.2 2020 Covid Crash: A Training Data Asset . . . . . 34

**IV Appendices** **35**

**8 Database Setup and Operations** **36**
8.1 PostgreSQL Installation and Configuration . . . . . 36
8.2 Master Orchestration Script . . . . . 37

**9 Data Quality Checklist** **39**

**10 Glossary of Key Terms** **41**

**Final Note** **43**

# Part I

# Theoretical Foundation

# Chapter 1

# The M/G/$\infty$ Queue and the Square-Root Rule

## 1.1 Why Queuing Theory Belongs in Finance

A portfolio manager running a valuation-based mean-reversion strategy faces a problem that is structurally identical to a service-operations problem. Consider the following analogy:

| **Service Operations** | | **Quant Portfolio** |
|---|---|---|
| Delivery fleet (total capacity $C$) | $\Leftrightarrow$ | Total capital ($500M) |
| Geographic regions | $\Leftrightarrow$ | Market sectors |
| Package arrival rate $\lambda_i$ | $\Leftrightarrow$ | Buy-signal rate per sector |
| Delivery duration $E[S_i]$ | $\Leftrightarrow$ | Avg. holding period to mean-reversion |
| Truck stockout (no truck available) | $\Leftrightarrow$ | Capital stockout (no cash to deploy) |

In both cases, the system manager must pre-allocate capacity to regions/sectors such that (a) the probability of running out of capacity is equal across all regions/sectors, and (b) excess idle capacity is minimized.

## 1.2 The M/G/$\infty$ Model

### 1.2.1 Model Specification

The M/G/$\infty$ queue makes the following assumptions:

- **M (Markovian arrivals):** Buy signals arrive according to a Poisson process with rate $\lambda_i$ for sector $i$. This is the standard assumption for exogenous, unsynchronized event streams.
- **G (General service):** Holding periods follow an arbitrary distribution—we only require the mean $E[S_i] = \mu_i^{-1}$. This is critical: we do not assume normally distributed holding times. Mean-reversion can take 40 days or 400 days.
- **$\infty$ (Infinite servers):** Every arriving signal that has capital allocated to it is served immediately. There is no queue—only a capital stockout if the budget is exceeded.

### 1.2.2 Steady-State Distribution

Under these assumptions, the number of positions simultaneously open in sector $i$ at any time, denoted $N_i$, follows a Poisson distribution:

$$N_i \sim \text{Poisson}(R_i), \quad \text{where} \quad R_i = \lambda_i \cdot E[S_i] \tag{1.1}$$

$R_i$ is the *offered load* of sector $i$—the expected number of open positions at any given moment. This is a direct consequence of Little's Law:

$$L = \lambda W \implies R_i = \lambda_i \cdot \mu_i^{-1} \tag{1.2}$$

### 1.2.3 The Capital Buffer Problem

Because $N_i \sim \text{Poisson}(R_i)$, the variance equals the mean:

$$\text{Var}(N_i) = R_i \tag{1.3}$$

The portfolio manager sets a target service level: the probability that the number of simultaneous open positions exceeds the allocated capital slots $c_i$ should be at most $\alpha$ (e.g., $\alpha = 1\%$):

$$P(N_i > c_i) \leq \alpha \quad \forall\, i \tag{1.4}$$

## 1.3 The Square-Root Allocation Rule

### 1.3.1 Derivation

For large $R_i$, the Poisson distribution is well-approximated by a Normal distribution. Using the Normal approximation:

$$P(N_i > c_i) = P\left(Z > \frac{c_i - R_i}{\sqrt{R_i}}\right) \leq \alpha \tag{1.5}$$

where $Z \sim \mathcal{N}(0,1)$. Let $k = \Phi^{-1}(1-\alpha)$ be the $z$-score for the target service level. Then:

$$\frac{c_i - R_i}{\sqrt{R_i}} = k \implies \boxed{c_i = R_i + k\sqrt{R_i}} \tag{1.6}$$

This is the **Square-Root Rule**. The key insight: $k$ is *universal across all sectors*. The same $k$ gives the same stockout probability in every sector, regardless of how different the $\lambda_i$ and $\mu_i$ values are.

### 1.3.2 Sector Budgeting

Given total capital $C$ (in normalized units), the universal $k$ is determined by the constraint:

$$\sum_{i=1}^{5} c_i = C \implies \sum_{i=1}^{5} \left(R_i + k\sqrt{R_i}\right) = C \tag{1.7}$$

Solving for $k$:

$$k = \frac{C - \sum_{i=1}^{5} R_i}{\sum_{i=1}^{5} \sqrt{R_i}} \tag{1.8}$$

Once $k$ is determined, apply the Square-Root Rule to each sector to get the capital allocation $c_i$.

### 1.3.3 Numerical Example

Suppose the five sectors have the following calibrated parameters:

| **Sector** | $\lambda_i$ (signals/yr) | $E[S_i]$ (yrs) | $R_i$ | $\sqrt{R_i}$ |
|---|---|---|---|---|
| Technology | 30 | 0.33 | 10.0 | 3.16 |
| Healthcare | 18 | 0.50 | 9.0 | 3.00 |
| Financials | 12 | 0.75 | 9.0 | 3.00 |
| Industrials | 8 | 0.60 | 4.8 | 2.19 |
| Consumer Goods | 10 | 0.40 | 4.0 | 2.00 |
| **Total** | | | **36.8** | **13.35** |

With $C = 50$ capital units and $\alpha = 1\%$ (so $k$ is determined by the formula above):

$$k = \frac{50 - 36.8}{13.35} = \frac{13.2}{13.35} \approx 0.989 \tag{1.9}$$

Capital allocations:

$$\begin{aligned}
c_{\text{Tech}} &= 10.0 + 0.989 \times 3.16 = 13.1 \text{ units} \\
c_{\text{HC}} &= 9.0 + 0.989 \times 3.00 = 11.97 \text{ units} \\
c_{\text{Fin}} &= 9.0 + 0.989 \times 3.00 = 11.97 \text{ units} \\
c_{\text{Ind}} &= 4.8 + 0.989 \times 2.19 = 6.97 \text{ units} \\
c_{\text{CG}} &= 4.0 + 0.989 \times 2.00 = 5.98 \text{ units} \\
\textbf{Total} &= \mathbf{49.99} \approx 50\checkmark
\end{aligned}$$

## 1.4 Why Valuation-Based Signals, Not Price

Price-based signals (moving average crossovers, RSI) are stationary by construction—they have no economic foundation for mean-reversion. Valuation-based signals, by contrast, have a fundamental anchor:

> **The Graham Valuation Principle.** A stock's intrinsic value is determined by its earnings power and asset base, not its recent price trajectory. When a stock's P/E (or P/B, EV/EBITDA) falls more than 2 standard deviations below its own historical mean, there is a quantifiable margin of safety. The market is mispricing the asset relative to its fundamental generating capacity. Mean-reversion here is not a statistical artifact—it is an economic expectation.

The signal process: for stock $j$ in sector $i$ on date $t$,

$$\text{Signal}_{j,t} = \mathbf{1}\left[\text{PE}_{j,t} < \overline{\text{PE}}_{j,\tau} - 2\sigma_{\text{PE},j,\tau}\right] \tag{1.10}$$

where $\overline{\text{PE}}_{j,\tau}$ and $\sigma_{\text{PE},j,\tau}$ are the rolling mean and standard deviation of stock $j$'s P/E ratio over the trailing window $\tau$ (typically 5 years). The arrival rate $\lambda_i$ is then the count of such signals in sector $i$ per unit time.

# Part II

# Data Infrastructure

# Chapter 2

# Universe Construction: Fixed Russell 3000 (2025)

## 2.1 Design Decision: Fixed Universe

**Known Limitation.** The approach in this chapter introduces survivorship bias by design. The universe is fixed to companies that exist in the Russell 3000 as of June 2025. Companies that failed, were acquired, or were delisted between 2010 and 2025 are absent. For a mathematical prototype focused on validating the M/G/$\infty$ allocation engine, this is acceptable. It is not acceptable for production use.

The universe source is the iShares IWV ETF holdings file (`IWV_holdings25.csv`), downloaded directly from iShares (free, no account required). The file contains one row per holding with the following columns already populated for us:

| CSV Column | Content | We Use It For |
|---|---|---|
| `Ticker` | e.g. `MSFT`, `AAPL` | Primary key everywhere |
| `Name` | e.g. `MICROSOFT CORP` | `company_name` in DB |
| `Sector` | GICS Level 1 (11 sectors) | Sector buckets for M/G/$\infty$ |
| `Asset Class` | `Equity`, `Futures`... | Filter: keep `Equity` only |
| `Exchange` | `NYSE`, `NASDAQ`... | Metadata |
| `Weight (%)` | ETF weight at snapshot date | Optional: market-cap proxy |

Everything else we need—prices and fundamentals—comes from `yfinance` in later phases.

## 2.2 Parsing `IWV_holdings25.csv`

The file has 9 metadata rows before the real header, and 6 non-equity rows (futures, cash, margin) plus 2 footer rows that must be stripped. After cleaning, we get **2,645 clean equity tickers**.

```python
import pandas as pd

def load_iwv_universe(filepath: str = "IWV_holdings25.csv") -> pd.DataFrame:
    """
    Parse the iShares IWV holdings CSV into a clean universe DataFrame.

```

```
    The file structure (verified against Jun 2025 download):
      Rows 0-8   : iShares metadata (fund name, date, totals)
      Row 9      : Column headers  (Ticker, Name, Sector, Asset Class, ...)
      Row 10+    : Holdings data
      Last 2     : NaN row + BlackRock legal disclaimer -- auto-dropped

    Returns a DataFrame with columns:
      ticker, company_name, sector, exchange
    """
    # skiprows=9 skips metadata; row 9 becomes the header
    df = pd.read_csv(filepath, skiprows=9, header=0)

    # --- Filter 1: Keep equity holdings only ---
    # Removes: Futures (ESU5, RTYU5), Cash, Cash Collateral, Money Market
    df = df[df["Asset Class"] == "Equity"].copy()

    # --- Filter 2: Keep only valid US equity tickers ---
    # Removes: "-" (unlisted warrants/CVRs like OMNIAB),
    #          "P5N994" (Petrocorp escrow - no market), NaN footer rows
    df = df[df["Ticker"].notna()]
    df = df[df["Ticker"].str.match(r'^[A-Z]{1,5}$', na=False)]

    # --- Rename to our DB column names ---
    universe = df[["Ticker", "Name", "Sector", "Exchange"]].copy()
    universe.columns = ["ticker", "company_name", "sector", "exchange"]
    universe = universe.drop_duplicates(subset="ticker").reset_index(drop=
   True)

    print(f"Universe loaded: {len(universe)} clean equity tickers")
    print(f"Sectors: {universe['sector'].nunique()} GICS sectors")
    return universe

UNIVERSE = load_iwv_universe("IWV_holdings25.csv")
# Output:
# Universe loaded: 2645 clean equity tickers
# Sectors: 11 GICS sectors
```

Listing 2.1: Parsing IWV_holdings25.csv into a Clean Universe DataFrame

**What You Get For Free From The CSV.** Unlike the original plan that required a yfinance enrichment pass to get sector and company name, the IWV CSV already provides `Ticker`, `Name`, `Sector`, and `Exchange` for all 2,645 tickers. You can load the entire metadata table in one shot from this file with no additional API calls.

## 2.3 Fundamental Data Strategy: yfinance vs. SEC EDGAR

**Honest Assessment of Fundamentals Complexity.** The original plan proposed fetching fundamentals from SEC EDGAR's XBRL API (true Point-in-Time correctness). This is the right long-term approach but is genuinely hard as a first project: you must resolve CIK numbers for all 2,645 tickers, handle missing XBRL tags, normalise different accounting taxonomies, and deal with amended filings. For a first build, `yfinance` quarterly financials are strongly recommended as a pragmatic starting point.

`yfinance` exposes quarterly financial statements going back several years via three DataFrames per ticker:

| yfinance attribute | What it contains | Key fields we extract |
|---|---|---|
| `.quarterly_financials` | Income statement (quarterly) | Net Income, Total Revenue, EPS |
| `.quarterly_balance_sheet` | Balance sheet (quarterly) | Total Assets, Total Equity, Debt |
| `.quarterly_cashflow` | Cash flow statement | Operating CF, CapEx, Free CF |

**The key limitation:** `yfinance` returns data *as it exists today*, not as it was originally filed. It does not tag a `filing_date`. This means fundamentals are not strictly PiT-correct—you are implicitly treating the period end date as the availability date. For a mean-reversion backtest where signals reset quarterly, this introduces a modest look-ahead bias (up to ∼60 days average filing lag). Accept this for version 1 and document it clearly.

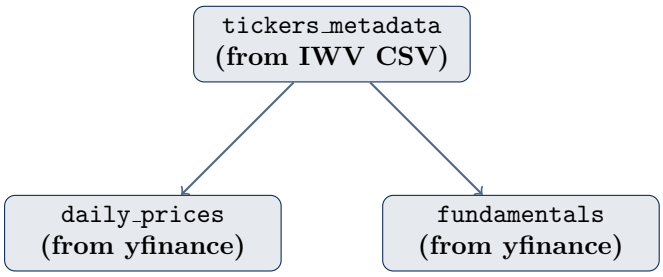
**PiT Rule (Relaxed, v1).** In this implementation, a fundamental value for period ending `period_end_date` is treated as available on that same date. In reality it becomes available 40–75 days later (the SEC filing deadline). Your WFO results will therefore be slightly optimistic. The fix—tagging with real `filing_date`s from EDGAR—is a clean future upgrade without changing any other part of the system.

# Chapter 3

# PostgreSQL Database Schema

## 3.1 Architecture Overview

Three tables are all we need for this version. The schema is deliberately minimal: every column in it is either (a) loaded directly from the IWV CSV, (b) fetched by `yfinance`, or (c) derived from those two sources. Nothing more.




**What we dropped from the original schema and why:**

- `index_constituents` / `static_universe` — redundant; `tickers_metadata` *is* the universe.
- `corporate_actions` — unnecessary because `yfinance` returns `adjusted_close` already split- and dividend-adjusted. We never need to apply factors ourselves.

## 3.2 Table Definitions

### 3.2.1 Table 1: `tickers_metadata` — The Master Roster

Every column in this table comes directly from the IWV CSV. No API calls required to populate it.

```sql
CREATE TABLE tickers_metadata (
    ticker          VARCHAR(12)  PRIMARY KEY,
    company_name    VARCHAR(255) NOT NULL,          -- CSV: "Name"
    sector          VARCHAR(64),                    -- CSV: "Sector" (GICS Level 1)
    exchange        VARCHAR(32),                    -- CSV: "Exchange"
    universe_type   VARCHAR(32)  NOT NULL DEFAULT 'FIXED_2025',
    snapshot_date   DATE         NOT NULL,          -- Jun 18, 2025 for this file
    created_at      TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);

```

```sql
CREATE INDEX idx_meta_sector ON tickers_metadata(sector);
```

Listing 3.1: Master Roster Schema — populated directly from IWV CSV

**No CIK column.** The original schema used CIK as the permanent identifier for joining XBRL data. Since we are fetching fundamentals via `yfinance` (which uses ticker symbols, not CIKs), we do not need a CIK column at all in v1. If you upgrade to EDGAR XBRL later, add a nullable `cik VARCHAR(12)` column at that time.

### 3.2.2 Table 2: `daily_prices` — Price History

End-of-day price data. The `adjusted_close` column is the critical column for all calculations—it accounts for splits and dividends and is the only price series that correctly represents actual investor returns.

```sql
CREATE TABLE daily_prices (
    price_id        BIGSERIAL    PRIMARY KEY,
    date            DATE         NOT NULL,
    ticker          VARCHAR(12)  REFERENCES tickers_metadata(ticker),
    open            NUMERIC(14,4),
    high            NUMERIC(14,4),
    low             NUMERIC(14,4),
    close           NUMERIC(14,4)  NOT NULL,
    adjusted_close  NUMERIC(14,4)  NOT NULL,  -- ALWAYS use this for calcs
    volume          BIGINT,
    UNIQUE (date, ticker)
);

CREATE INDEX idx_prices_ticker_date ON daily_prices(ticker, date);
CREATE INDEX idx_prices_date        ON daily_prices(date);
```

Listing 3.2: Daily Prices Schema

**Unadjusted Price Bias.** If a stock splits 2-for-1, its unadjusted price halves overnight. A naive algorithm sees a 50% price drop and triggers a massive Buy signal. This is entirely spurious. Only `adjusted_close`, which retroactively adjusts all historical prices for splits and dividends, produces economically meaningful return calculations. Never use raw `close` for strategy calculations.

### 3.2.3 Table 3: `fundamentals` — Quarterly Financials

Each row is one quarterly snapshot for one ticker, fetched from `yfinance`. The `period_end_date` is the fiscal quarter end. No `filing_date` column—this is the v1 relaxed PiT approach discussed in Chapter 3.

```sql
CREATE TABLE fundamentals (
    fundamental_id   BIGSERIAL   PRIMARY KEY,
    ticker           VARCHAR(12) REFERENCES tickers_metadata(ticker),
    period_end_date  DATE        NOT NULL,  -- Fiscal quarter end date

    -- Income Statement
    revenue          NUMERIC(18,2),
    gross_profit     NUMERIC(18,2),
    net_income       NUMERIC(18,2),
    eps_diluted      NUMERIC(12,4),

```

```sql
    -- Balance Sheet
    total_assets      NUMERIC(18,2),
    total_equity      NUMERIC(18,2),
    long_term_debt    NUMERIC(18,2),

    -- Cash Flow
    operating_cf      NUMERIC(18,2),    -- Operating cash flow
    capex             NUMERIC(18,2),

    -- Derived (compute on insert)
    book_value_per_share NUMERIC(12,4),  -- total_equity /
        shares_outstanding
    free_cash_flow    NUMERIC(18,2),      -- operating_cf - capex

    source            VARCHAR(16)  DEFAULT 'YFINANCE',
    created_at        TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,

    UNIQUE (ticker, period_end_date)
);

CREATE INDEX idx_fund_ticker_date ON fundamentals(ticker, period_end_date);

-- Query Pattern: "Most recent fundamentals for AAPL as of 2018-01-15"
-- (relaxed PiT: treats period_end_date as availability date)
-- SELECT * FROM fundamentals
-- WHERE ticker = 'AAPL'
--   AND period_end_date <= '2018-01-15'
-- ORDER BY period_end_date DESC
-- LIMIT 1;
```

Listing 3.3: Simplified Fundamentals Schema (yfinance-sourced)

# Chapter 4

# Data Ingestion Pipeline

## 4.1 Phase 1: Universe Loading from IWV CSV

One script, one source, one DB insert. Every column in `tickers_metadata` comes directly from `IWV_holdings25.csv`.

```python
import psycopg2
from psycopg2.extras import execute_values
import pandas as pd

def load_iwv_universe(filepath: str = "IWV_holdings25.csv") -> pd.DataFrame:
    """Parse IWV_holdings25.csv into a clean universe DataFrame."""
    df = pd.read_csv(filepath, skiprows=9, header=0)
    df = df[df["Asset Class"] == "Equity"].copy()
    df = df[df["Ticker"].notna()]
    df = df[df["Ticker"].str.match(r'^[A-Z]{1,5}$', na=False)]
    universe = df[["Ticker", "Name", "Sector", "Exchange"]].copy()
    universe.columns = ["ticker", "company_name", "sector", "exchange"]
    return universe.drop_duplicates(subset="ticker").reset_index(drop=True)

def insert_universe_to_db(universe: pd.DataFrame):
    conn = psycopg2.connect(
        dbname="quant_db", user="quant_user",
        password="your_password", host="localhost", port=5432
    )
    cur = conn.cursor()

    rows = [
        (row["ticker"], row["company_name"], row["sector"],
         row["exchange"], "FIXED_2025", "2025-06-18")
        for _, row in universe.iterrows()
    ]

    cur.execute("""
        CREATE TABLE IF NOT EXISTS tickers_metadata (
            ticker          VARCHAR(12)  PRIMARY KEY,
            company_name    VARCHAR(255) NOT NULL,
            sector          VARCHAR(64),
            exchange        VARCHAR(32),
            universe_type   VARCHAR(32)  NOT NULL DEFAULT 'FIXED_2025',
            snapshot_date   DATE         NOT NULL,
            created_at      TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
        )
    """)

```

```
    execute_values(cur, """
        INSERT INTO tickers_metadata
            (ticker, company_name, sector, exchange, universe_type,
    snapshot_date)
        VALUES %s
        ON CONFLICT (ticker) DO NOTHING
    """, rows)

    conn.commit()
    print(f"Inserted {len(rows)} tickers into tickers_metadata")
    cur.close()
    conn.close()

universe = load_iwv_universe("IWV_holdings25.csv")
insert_universe_to_db(universe)
# Output: Inserted 2645 tickers into tickers_metadata
```

Listing 4.1: Loading IWV CSV Directly into PostgreSQL

## 4.2 Phase 2: Price Data via yfinance

`yfinance`'s `download()` function fetches adjusted daily closes for a list of tickers in batched requests. For 2,645 tickers fetching from 2010–01–01 to present, this takes roughly **30–90 minutes** depending on network speed. Run it once and never again.

```
import yfinance as yf
import psycopg2
from psycopg2.extras import execute_values
import pandas as pd
from datetime import date

def bulk_download_prices(tickers: list[str],
                         start: str = "2010-01-01",
                         batch_size: int = 100):
    """
    Download adjusted daily closes for all tickers, in batches of 100.
    yfinance handles batches internally but explicit batching gives
    cleaner error recovery if a batch fails.
    """
    conn = psycopg2.connect(
        dbname="quant_db", user="quant_user",
        password="your_password", host="localhost", port=5432
    )
    cur = conn.cursor()

    cur.execute("""
        CREATE TABLE IF NOT EXISTS daily_prices (
            date            DATE          NOT NULL,
            ticker          VARCHAR(12)   NOT NULL,
            adj_close       NUMERIC(14,4) NOT NULL,
            volume          BIGINT,
            PRIMARY KEY (date, ticker)
        )
    """)
    conn.commit()

    total_batches = (len(tickers) + batch_size - 1) // batch_size

    for i in range(0, len(tickers), batch_size):
        batch = tickers[i : i + batch_size]
        batch_num = i // batch_size + 1
        print(f"Batch {batch_num}/{total_batches}: {len(batch)} tickers")
```

```python

        try:
            raw = yf.download(
                batch,
                start=start,
                end=str(date.today()),
                auto_adjust=True,   # Returns adjusted prices; 'Close' IS adj_close
                progress=False
            )
        except Exception as e:
            print(f"  ERROR on batch {batch_num}: {e}")
            continue

        # yfinance multi-ticker download has a MultiIndex:
        # columns = (metric, ticker)
        # We only need 'Close' (which is adjusted when auto_adjust=True)
        if isinstance(raw.columns, pd.MultiIndex):
            close_df = raw["Close"]   # shape: (dates, tickers)
            vol_df   = raw["Volume"]
        else:
            # Single ticker fallback
            close_df = raw[["Close"]].rename(columns={"Close": batch[0]})
            vol_df   = raw[["Volume"]].rename(columns={"Volume": batch[0]})

        rows = []
        for ticker in close_df.columns:
            series = close_df[ticker].dropna()
            vols   = vol_df[ticker] if ticker in vol_df.columns else None
            for dt, price in series.items():
                vol = int(vols[dt]) if vols is not None and dt in vols.index else None
                rows.append((dt.date(), ticker, float(price), vol))

        if rows:
            execute_values(cur, """
                INSERT INTO daily_prices (date, ticker, adj_close, volume)
                VALUES %s
                ON CONFLICT (date, ticker) DO NOTHING
            """, rows)
            conn.commit()
            print(f"  Inserted {len(rows):,} price rows")

    cur.close()
    conn.close()
    print("Price download complete.")

# Run it:
TICKERS = pd.read_sql(
    "SELECT ticker FROM tickers_metadata ORDER BY ticker",
    con=psycopg2.connect(dbname="quant_db", user="quant_user",
                         password="your_password", host="localhost")
)["ticker"].tolist()

bulk_download_prices(TICKERS, start="2010-01-01")
```

Listing 4.2: Bulk Price Download via yfinance (2010–Present)

**Missing Price History for Young Companies.** Companies that IPO'd after 2010 will simply have shorter price history—`yfinance` returns whatever data exists. This is fine; the WFO engine will exclude a ticker from any window that starts before its IPO

date (it will have no data for that period). No special handling required.

## 4.3 Phase 3: Fundamental Data via yfinance

`yfinance` exposes quarterly financials via three per-ticker DataFrames. The challenge: unlike prices (where `download()` is batched), fundamentals must be fetched one ticker at a time. At ∼0.5–1 second per ticker, **2,645 tickers will take 45–90 minutes**. Run it overnight.

```python
import yfinance as yf
import psycopg2
from psycopg2.extras import execute_values
import pandas as pd
import time

def extract_yf_fundamentals(ticker: str) -> list[dict]:
    """
    Fetch quarterly income statement, balance sheet, and cash flow
    for one ticker from yfinance. Returns a list of dicts,
    one per quarter.
    """
    t = yf.Ticker(ticker)

    try:
        inc = t.quarterly_financials    # columns = quarter end dates
        bal = t.quarterly_balance_sheet
        cf  = t.quarterly_cashflow
    except Exception:
        return []

    if inc is None or inc.empty:
        return []

    records = []
    for period in inc.columns:  # Each column is a quarter-end date
        def safe_get(df, row):
            try:
                return float(df.loc[row, period]) if row in df.index else None
            except Exception:
                return None

        rec = {
            "ticker":          ticker,
            "period_end_date": period.date(),
            "revenue":         safe_get(inc, "Total Revenue"),
            "gross_profit":    safe_get(inc, "Gross Profit"),
            "net_income":      safe_get(inc, "Net Income"),
            "eps_diluted":     safe_get(inc, "Diluted EPS"),
            "total_assets":    safe_get(bal, "Total Assets"),
            "total_equity":    safe_get(bal, "Stockholders Equity"),
            "long_term_debt":  safe_get(bal, "Long Term Debt"),
            "operating_cf":    safe_get(cf,  "Operating Cash Flow"),
            "capex":           safe_get(cf,  "Capital Expenditure"),
        }
        # Derived fields
        rec["free_cash_flow"] = (
            (rec["operating_cf"] or 0) + (rec["capex"] or 0)
            # yfinance reports capex as negative, so adding gives FCF
        )
        records.append(rec)

```

```python
    return records

def load_all_fundamentals(tickers: list[str]):
    conn = psycopg2.connect(
        dbname="quant_db", user="quant_user",
        password="your_password", host="localhost", port=5432
    )
    cur = conn.cursor()

    cur.execute("""
        CREATE TABLE IF NOT EXISTS fundamentals (
            fundamental_id  BIGSERIAL     PRIMARY KEY,
            ticker          VARCHAR(12)   REFERENCES tickers_metadata(ticker),
            period_end_date DATE          NOT NULL,
            revenue         NUMERIC(18,2),
            gross_profit    NUMERIC(18,2),
            net_income      NUMERIC(18,2),
            eps_diluted     NUMERIC(12,4),
            total_assets    NUMERIC(18,2),
            total_equity    NUMERIC(18,2),
            long_term_debt  NUMERIC(18,2),
            operating_cf    NUMERIC(18,2),
            capex           NUMERIC(18,2),
            free_cash_flow  NUMERIC(18,2),
            source          VARCHAR(16)   DEFAULT 'YFINANCE',
            created_at      TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
            UNIQUE (ticker, period_end_date)
        )
    """)
    conn.commit()

    failed = []
    for i, ticker in enumerate(tickers):
        if i % 100 == 0:
            print(f"Progress: {i}/{len(tickers)} ({i/len(tickers)*100:.1f}%)")

        records = extract_yf_fundamentals(ticker)
        if not records:
            failed.append(ticker)
            continue

        rows = [(
            r["ticker"], r["period_end_date"],
            r["revenue"], r["gross_profit"], r["net_income"], r["eps_diluted"],
            r["total_assets"], r["total_equity"], r["long_term_debt"],
            r["operating_cf"], r["capex"], r["free_cash_flow"]
        ) for r in records]

        execute_values(cur, """
            INSERT INTO fundamentals
                (ticker, period_end_date, revenue, gross_profit, net_income,
                 eps_diluted, total_assets, total_equity, long_term_debt,
                 operating_cf, capex, free_cash_flow)
            VALUES %s
            ON CONFLICT (ticker, period_end_date) DO NOTHING
        """, rows)
        conn.commit()
        time.sleep(0.3)  # Gentle on yfinance rate limits

```

```
    print(f"Done. Failed tickers ({len(failed)}): {failed[:20]}")
    cur.close()
    conn.close()

load_all_fundamentals(TICKERS)
```

Listing 4.3: Quarterly Fundamentals via yfinance (One Ticker at a Time)

**yfinance Fundamental Coverage.** yfinance typically returns 4–5 years of quarterly history per ticker (going back to around 2019–2020). It does **not** provide 15 years of quarterly fundamentals back to 2010 for most stocks. This means your WFO windows before ∼2019 will have no fundamental signal and cannot generate buy signals. In practice this means your effective backtest window with fundamentals is **2020–2025**, not 2010–2025. The price data still covers the full window, which is useful for benchmarking returns once a signal fires. If you need pre-2019 fundamentals you must switch to SEC EDGAR XBRL.

# Part III

# Strategy Implementation

# Chapter 5

# Extracting M/G/∞ Parameters from the Database

## 5.1 The Valuation Signal

With the database populated, parameter extraction follows a straightforward sequence. All queries use the three tables: `tickers_metadata`, `daily_prices`, and `fundamentals`.

**Relaxed PiT in These Queries.** Because fundamentals use `period_end_date` as the availability date (not a real `filing_date`), the queries below filter on `period_end_date <= simulation_date`. This is the v1 relaxation documented in Chapter 3.

### 5.1.1 Computing P/E Ratios at a Simulation Date

```python
import pandas as pd
import numpy as np
import psycopg2

DB_CONFIG = {
    "dbname": "quant_db", "user": "quant_user",
    "password": "your_password", "host": "localhost", "port": 5432
}

def compute_pe(simulation_date: str) -> pd.DataFrame:
    """
    For each stock in the universe, compute P/E at simulation_date using:
      - Most recent adj_close on or before simulation_date
      - Most recent quarterly EPS with period_end_date <= simulation_date

    Returns DataFrame: ticker, sector, price, eps_diluted, pe_ratio
    """
    conn = psycopg2.connect(**DB_CONFIG)

    # Most recent price per ticker
    price_query = """
        SELECT DISTINCT ON (ticker)
            ticker, adj_close AS price
        FROM daily_prices
        WHERE date <= %(sim_date)s
        ORDER BY ticker, date DESC
    """

```

```
    # Most recent EPS per ticker (relaxed PiT: period_end_date as availability)
    eps_query = """
        SELECT DISTINCT ON (ticker)
            ticker, eps_diluted, period_end_date
        FROM fundamentals
        WHERE period_end_date <= %(sim_date)s
          AND eps_diluted IS NOT NULL
          AND eps_diluted > 0
        ORDER BY ticker, period_end_date DESC
    """

    prices = pd.read_sql(price_query, conn, params={"sim_date": simulation_date})
    eps    = pd.read_sql(eps_query,   conn, params={"sim_date": simulation_date})
    meta   = pd.read_sql("SELECT ticker, sector FROM tickers_metadata", conn)
    conn.close()

    df = meta.merge(prices, on="ticker", how="inner")
    df = df.merge(eps,    on="ticker", how="inner")
    df["pe_ratio"] = df["price"] / df["eps_diluted"]

    # Remove outliers (data quality gate)
    df = df[(df["pe_ratio"] > 1) & (df["pe_ratio"] < 200)]
    return df
```

Listing 5.1: P/E Ratio Calculation (Relaxed PiT)

### 5.1.2 Generating Signals: The 2-SD Threshold

```
def generate_signals(
    sim_date:      str,
    lookback_days: int = 504  # ~2 trading years (realistic with yfinance history)
) -> pd.DataFrame:
    """
    For each stock, check if P/E on sim_date is >=2 SD below its
    own rolling mean over the lookback window.
    Relaxed PiT: joins on period_end_date <= dp.date (not filing_date).
    """
    conn = psycopg2.connect(**DB_CONFIG)

    hist_query = """
        WITH daily_eps AS (
            SELECT
                dp.date,
                dp.ticker,
                dp.adj_close,
                f.eps_diluted
            FROM daily_prices dp
            CROSS JOIN LATERAL (
                SELECT eps_diluted
                FROM fundamentals
                WHERE ticker = dp.ticker
                  AND period_end_date <= dp.date
                  AND eps_diluted > 0
                ORDER BY period_end_date DESC
                LIMIT 1
            ) f
            WHERE dp.date BETWEEN %(start_date)s AND %(sim_date)s
        )
```

```
        SELECT
            de.date,
            de.ticker,
            tm.sector,
            de.adj_close / de.eps_diluted AS pe_ratio
        FROM daily_eps de
        JOIN tickers_metadata tm ON de.ticker = tm.ticker
        ORDER BY de.ticker, de.date
    """

    from datetime import datetime, timedelta
    start = (datetime.strptime(sim_date, '%Y-%m-%d')
             - timedelta(days=lookback_days * 1.5)).strftime('%Y-%m-%d')

    hist = pd.read_sql(
        hist_query, conn,
        params={'start_date': start, 'sim_date': sim_date}
    )
    conn.close()

    results = []
    for ticker, grp in hist.groupby('ticker'):
        grp = grp.sort_values('date').tail(lookback_days)
        if len(grp) < 126:  # Need at least 6 months of history
            continue

        rolling_mean = grp['pe_ratio'].mean()
        rolling_std  = grp['pe_ratio'].std()
        if rolling_std == 0:
            continue

        current_pe = grp.iloc[-1]['pe_ratio']
        z_score    = (current_pe - rolling_mean) / rolling_std
        results.append({
            'ticker':       ticker,
            'sector':       grp.iloc[-1]['sector'],
            'current_pe':   current_pe,
            'rolling_mean': rolling_mean,
            'rolling_std':  rolling_std,
            'z_score':      z_score,
            'is_signal':    z_score <= -2.0
        })

    df      = pd.DataFrame(results)
    signals = df[df['is_signal']].copy()
    print(f"Signals on {sim_date}: {len(signals)} across {signals['sector
    ].nunique()} sectors")
    return signals
```

Listing 5.2: Rolling Valuation Signal Detection (Relaxed PiT)

## 5.2 Computing $\lambda_i$ and $\mu_i$ from Historical Data

To calibrate the M/G/∞ queue, we need the signal arrival rate $\lambda_i$ and mean holding duration $\mu_i^{-1}$ for each sector $i$.

```
from scipy.stats import norm

def compute_mginfinity_params(
    train_start: str,
    train_end:   str,
    lookback_window_days: int = 1260
```

```
) -> dict:
    """
    Compute lambda_i and mu_i for each sector using training period data.

    lambda_i = number of 2-SD buy signals per year in sector i
    mu_i^{-1} = average days for P/E to revert to mean after signal

    Returns: {sector: {'lambda': float, 'mu_inv': float, 'R': float}}
    """
    import pandas as pd
    from datetime import datetime, timedelta

    train_start_dt = datetime.strptime(train_start, '%Y-%m-%d')
    train_end_dt   = datetime.strptime(train_end,   '%Y-%m-%d')
    years_in_sample = (train_end_dt - train_start_dt).days / 365.25

    # Step 1: Generate signals for every month in training period
    print("Generating historical signals for training period...")
    all_signals = []
    current = train_start_dt
    while current <= train_end_dt:
        sim_date = current.strftime('%Y-%m-%d')
        try:
            sigs = generate_signals(sim_date, lookback_days=
    lookback_window_days)
            sigs['signal_date'] = sim_date
            all_signals.append(sigs)
        except Exception as e:
            print(f"  Skipping {sim_date}: {e}")
        current += timedelta(days=21)  # Monthly sampling

    all_signals_df = pd.concat(all_signals, ignore_index=True)

    # Step 2: Compute lambda (signals per year per sector)
    signal_counts = (
        all_signals_df[all_signals_df['is_signal']]
        .groupby('sector')['ticker']
        .count()
    )
    lambda_i = (signal_counts / years_in_sample).to_dict()

    # Step 3: Compute mean holding duration (mu^{-1}) per sector
    # For each triggered signal, find how many days until P/E reverts above
     mean
    print("Computing mean reversion holding durations...")
    reversion_durations = {}
    conn = psycopg2.connect(**DB_CONFIG)

    triggered = all_signals_df[all_signals_df['is_signal']].copy()
    triggered = triggered.drop_duplicates(subset=['ticker', 'signal_date'])

    for _, row in triggered.iterrows():
        ticker      = row['ticker']
        sector      = row['sector']
        signal_date = row['signal_date']
        mean_pe     = row['rolling_mean']

        # Find first date after signal where P/E returns above rolling mean
        revert_query = """
            SELECT dp.date, dp.adj_close / f.eps_diluted AS pe_ratio
            FROM daily_prices dp
            CROSS JOIN LATERAL (
                SELECT eps_diluted FROM fundamentals
```

```
                WHERE ticker = dp.ticker
                  AND period_end_date <= dp.date
                  AND eps_diluted > 0
                ORDER BY period_end_date DESC LIMIT 1
            ) f
            WHERE dp.ticker = %(ticker)s
              AND dp.date > %(signal_date)s
              AND dp.date <= %(max_date)s
            ORDER BY dp.date
            LIMIT 500
        """
        max_date = (
            datetime.strptime(signal_date, '%Y-%m-%d') + timedelta(days=1500)
        ).strftime('%Y-%m-%d')

        try:
            future = pd.read_sql(
                revert_query, conn,
                params={'ticker': ticker, 'signal_date': signal_date,
                        'max_date': max_date}
            )

            reverted = future[future['pe_ratio'] >= mean_pe]
            if not reverted.empty:
                signal_dt  = datetime.strptime(signal_date, '%Y-%m-%d')
                revert_dt  = pd.to_datetime(reverted.iloc[0]['date'])
                duration_days = (revert_dt - signal_dt).days

                if sector not in reversion_durations:
                    reversion_durations[sector] = []
                reversion_durations[sector].append(duration_days)
        except Exception:
            pass

    conn.close()

    # mu^{-1} in years
    mu_inv_i = {
        sector: np.mean(durations) / 365.25
        for sector, durations in reversion_durations.items()
        if durations
    }

    # Combine into final parameter dict
    params = {}
    all_sectors = set(lambda_i.keys()) | set(mu_inv_i.keys())
    for sector in all_sectors:
        lam   = lambda_i.get(sector, 0)
        mu_inv = mu_inv_i.get(sector, 0.5)
        R = lam * mu_inv
        params[sector] = {
            'lambda':  lam,
            'mu_inv':  mu_inv,
            'R':       R,
            'sqrt_R':  np.sqrt(R) if R > 0 else 0
        }

    return params


def compute_capital_allocation(params: dict, total_capital: float) -> dict:
    """
```

```python
    Apply the Square-Root Rule to compute sector capital allocations.

    c_i = R_i + k * sqrt(R_i)

    k is the universal buffer scalar solved by: sum(c_i) = total_capital
    """
    sum_R     = sum(p['R']      for p in params.values())
    sum_sqrtR = sum(p['sqrt_R'] for p in params.values())

    if sum_sqrtR == 0:
        raise ValueError("No valid sectors with positive R values")

    k = (total_capital - sum_R) / sum_sqrtR

    allocations = {}
    for sector, p in params.items():
        c_i = p['R'] + k * p['sqrt_R']
        allocations[sector] = {
            'R':          p['R'],
            'k':          k,
            'c_i':        max(c_i, 0),  # Cannot be negative
            'lambda':     p['lambda'],
            'mu_inv_yrs': p['mu_inv'],
        }

    print(f"\nUniversal buffer scalar k = {k:.4f}")
    print(f"{'Sector':<20} {'R_i':>8} {'c_i':>10} {'Capital%':>10}")
    print("-" * 52)
    for sector, alloc in allocations.items():
        pct = alloc['c_i'] / total_capital * 100
        print(f"{sector:<20} {alloc['R']:>8.2f} {alloc['c_i']:>10.2f} {pct:>9.1f}%")

    return allocations
```

Listing 5.3: M/G/Infinity Parameter Extraction

# Chapter 6

# Walk-Forward Optimization

## 6.1 Methodology

Walk-Forward Optimization (WFO) is the gold standard for validating quantitative trading strategies. Instead of optimizing on one historical period and testing on another, WFO continuously recalibrates the model as new data arrives, simulating the exact experience of a live portfolio manager.

### 6.1.1 Window Configuration

| Parameter | Value | Rationale |
|---|---|---|
| Training window | 2 years (504 trading days) | Matches yfinance fundamental history depth |
| Test (OOS) window | 1 year (252 trading days) | Annual recalibration |
| First training start | January 2020 | First year with reliable yfinance fundamentals |
| First test start | January 2022 | First OOS period |
| Final test end | December 2025 | 3-year OOS evaluation |

**Why the window starts in 2020, not 2010.** yfinance returns approximately 4–5 years of quarterly fundamentals per ticker. The P/E signal requires at least 6 months of EPS history before it can fire. A 2020 start is therefore the earliest realistic first training date. Price data from 2010 is still in the database and is used for return calculations once a position opens—it is not wasted.

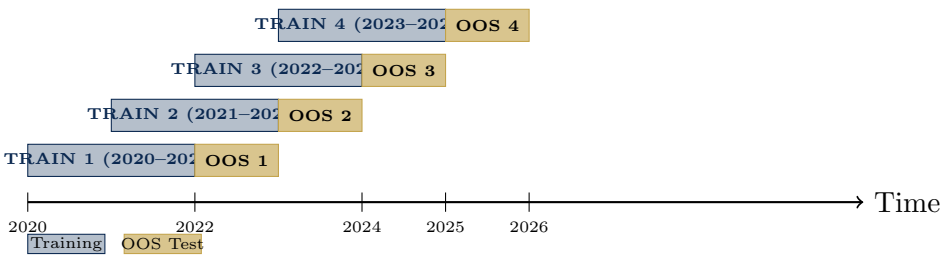



Figure 6.1: Walk-Forward Optimization Window Structure (v1: 2-year train, 1-year OOS). Four OOS windows covering 2022–2025. Parameters are recalibrated annually.

## 6.2 WFO Engine Implementation

```python
from datetime import datetime, timedelta
import json

def run_walk_forward_optimization(
    total_capital_usd:   float = 500_000_000,
    train_years:         int   = 2,
    oos_years:           int   = 1,
    wfo_start:           str   = "2020-01-01",
    wfo_end:             str   = "2025-12-31",
    target_alpha:        float = 0.01  # 1% stockout probability target
) -> pd.DataFrame:
    """
    Main Walk-Forward Optimization loop.
    Returns DataFrame of annual OOS results.
    """

    from scipy.stats import norm
    k_target = norm.ppf(1 - target_alpha)

    results = []
    current_year = datetime.strptime(wfo_start, '%Y-%m-%d').year + train_years

    while True:
        # Define windows
        oos_start = f"{current_year}-01-01"
        oos_end   = f"{current_year}-12-31"
        train_start = f"{current_year - train_years}-01-01"
        train_end   = f"{current_year - 1}-12-31"

        if datetime.strptime(oos_end, '%Y-%m-%d') > datetime.strptime(wfo_end, '%Y-%m-%d'):
            break

        print(f"\n{'='*60}")
        print(f"TRAINING: {train_start} to {train_end}")
        print(f"OUT-OF-SAMPLE: {oos_start} to {oos_end}")
        print(f"{'='*60}")

        # --- TRAINING PHASE: Calibrate M/G/inf parameters ---
        try:
            params = compute_mginfinity_params(train_start, train_end)
            allocations = compute_capital_allocation(
                params,
                total_capital=100.0  # Use 100 units; scale to USD later
            )
        except Exception as e:
            print(f"  TRAINING FAILED: {e}")
            current_year += oos_years
            continue

        # --- OOS PHASE: Simulate portfolio from oos_start to oos_end ---
        oos_metrics = simulate_portfolio_oos(
            allocations      = allocations,
            oos_start        = oos_start,
            oos_end          = oos_end,
            total_capital    = total_capital_usd,
            target_alpha     = target_alpha
        )

        # Record results
```

```
        results.append({
            'train_start':          train_start,
            'train_end':            train_end,
            'oos_year':             current_year,
            'total_return_pct':     oos_metrics['total_return_pct'],
            'sharpe_ratio':         oos_metrics['sharpe_ratio'],
            'max_drawdown_pct':     oos_metrics['max_drawdown_pct'],
            'avg_stockout_pct':     oos_metrics['avg_stockout_pct'],
            'signals_triggered':    oos_metrics['signals_triggered'],
            'signals_rejected':     oos_metrics['signals_rejected'],
            'sector_imbalance':     oos_metrics['sector_stockout_std'],
            **{f"alloc_{s[:4]}": allocations.get(s, {}).get('c_i', 0)
               for s in allocations}
        })

        print(f"\n  OOS RESULT {current_year}:")
        print(f"    Return:       {oos_metrics['total_return_pct']:.2f}%")
        print(f"    Sharpe:       {oos_metrics['sharpe_ratio']:.3f}")
        print(f"    Max DD:       {oos_metrics['max_drawdown_pct']:.2f}%")
        print(f"    Stockout Rate:{oos_metrics['avg_stockout_pct']:.2f}%")

        current_year += oos_years

    results_df = pd.DataFrame(results)
    return results_df


def simulate_portfolio_oos(
    allocations:   dict,
    oos_start:     str,
    oos_end:       str,
    total_capital: float,
    target_alpha:  float
) -> dict:
    """
    Simulate the M/G/inf portfolio over the OOS period.
    On each day:
      1. Check for new buy signals (P/E drops 2-SD below mean)
      2. Accept signal if sector has available capital (c_i not exhausted)
      3. Reject signal if sector is fully deployed
      4. Close positions when P/E reverts to rolling mean
      5. Track PnL, drawdown, and stockout events

    Returns dict of performance metrics.
    """
    conn = psycopg2.connect(**DB_CONFIG)

    # Load trading days in OOS period
    trading_days = pd.read_sql(
        "SELECT DISTINCT date FROM daily_prices "
        "WHERE date BETWEEN %(s)s AND %(e)s ORDER BY date",
        conn, params={'s': oos_start, 'e': oos_end}
    )['date'].tolist()
    conn.close()

    # Portfolio state
    open_positions = {}    # {ticker: {entry_price, entry_date, sector,
    target_pe}}
    sector_deployed = {s: 0 for s in allocations}  # Units of capital
    deployed
    daily_pnl = []
    stockout_events = 0
    signals_triggered = 0
```

```
    signals_rejected  = 0

    # Normalize capital: 1 unit = total_capital / 100
    capital_unit = total_capital / 100.0

    for sim_date in trading_days:
        sim_date_str = sim_date.strftime('%Y-%m-%d')

        # Check for exits: close positions where P/E has reverted
        positions_to_close = []
        conn = psycopg2.connect(**DB_CONFIG)
        for ticker, pos in open_positions.items():
            cur_pe_df = pd.read_sql(
                """
                SELECT dp.adj_close / f.eps_diluted AS pe
                FROM daily_prices dp
                CROSS JOIN LATERAL (
                    SELECT eps_diluted FROM fundamentals
                    WHERE ticker=dp.ticker
                      AND period_end_date<=dp.date
                      AND eps_diluted > 0
                    ORDER BY period_end_date DESC LIMIT 1
                ) f
                WHERE dp.ticker=%(t)s AND dp.date=%(d)s
                """,
                conn, params={'t': ticker, 'd': sim_date_str}
            )
            if cur_pe_df.empty:
                continue

            current_pe = cur_pe_df.iloc[0]['pe']
            if current_pe >= pos['target_pe']:  # P/E reverted to mean
                # Calculate PnL
                cur_price_df = pd.read_sql(
                    "SELECT adj_close FROM daily_prices "
                    "WHERE ticker=%(t)s AND date=%(d)s",
                    conn, params={'t': ticker, 'd': sim_date_str}
                )
                if not cur_price_df.empty:
                    ret = (cur_price_df.iloc[0]['adj_close']
                           / pos['entry_price']) - 1
                    daily_pnl.append(ret * capital_unit)

                positions_to_close.append(ticker)
                sector_deployed[pos['sector']] -= 1

        for t in positions_to_close:
            del open_positions[t]

        # Check for new signals
        try:
            signals = generate_signals(sim_date_str)
        except Exception:
            conn.close()
            continue

        for _, sig in signals.iterrows():
            ticker = sig['ticker']
            sector = sig['sector']
            if ticker in open_positions:
                continue  # Already in position

            signals_triggered += 1
```

```python

            # Check sector capacity (M/G/inf queue gate)
            max_slots = allocations.get(sector, {}).get('c_i', 0)
            deployed  = sector_deployed.get(sector, 0)

            if deployed >= max_slots:
                signals_rejected += 1  # STOCKOUT EVENT
                stockout_events  += 1
                continue

            # Open position
            entry_price_df = pd.read_sql(
                "SELECT adj_close FROM daily_prices "
                "WHERE ticker=%(t)s AND date=%(d)s",
                conn, params={'t': ticker, 'd': sim_date_str}
            )
            if entry_price_df.empty:
                continue

            open_positions[ticker] = {
                'entry_price': entry_price_df.iloc[0]['adj_close'],
                'entry_date':  sim_date_str,
                'sector':      sector,
                'target_pe':   sig['rolling_mean']
            }
            sector_deployed[sector] = sector_deployed.get(sector, 0) + 1

        conn.close()

    # Compute summary metrics
    if not daily_pnl:
        return {k: 0 for k in [
            'total_return_pct', 'sharpe_ratio', 'max_drawdown_pct',
            'avg_stockout_pct', 'signals_triggered', 'signals_rejected',
            'sector_stockout_std'
        ]}

    pnl_array   = np.array(daily_pnl)
    cum_returns = np.cumsum(pnl_array) / total_capital * 100
    total_ret   = cum_returns[-1] if len(cum_returns) > 0 else 0
    sharpe      = (np.mean(pnl_array) / (np.std(pnl_array) + 1e-10)) * np.sqrt(252)
    max_dd      = np.min(cum_returns - np.maximum.accumulate(cum_returns))
    stockout_pct = (stockout_events / max(signals_triggered, 1)) * 100

    return {
        'total_return_pct':   total_ret,
        'sharpe_ratio':       sharpe,
        'max_drawdown_pct':   max_dd,
        'avg_stockout_pct':   stockout_pct,
        'signals_triggered':  signals_triggered,
        'signals_rejected':   signals_rejected,
        'sector_stockout_std': np.std(list(sector_deployed.values()))
    }
```

Listing 6.1: Complete Walk-Forward Optimization Loop

## 6.3 Interpreting Walk-Forward Results

After running the full WFO loop, evaluate results against these institutional benchmarks:

| **Metric** | **Below Average** | **Acceptable** | **Institutional** |
|---|---|---|---|
| Annual Return | $< 5\%$ | 5%–12% | $> 12\%$ |
| Sharpe Ratio | $< 0.5$ | 0.5–1.0 | $> 1.0$ |
| Max Drawdown | $> -30\%$ | $-15\%$ to $-30\%$ | $< -15\%$ |
| Stockout Rate | $> 20\%$ | 5%–20% | $< 5\%$ |
| Sector Imbalance | High | Moderate | Near-zero |

The Stockout Rate is the M/G/∞-specific diagnostic. If it exceeds your target $\alpha$ systematically, your $\lambda_i$ estimates were too low during calibration—the market generated more signals than the model anticipated. The Square-Root Rule's buffer $k\sqrt{R_i}$ was insufficient. Solutions: increase the training window, add a safety multiplier to $k$, or re-examine the signal definition.

# Chapter 7

# Regime Analysis and Stress Testing

## 7.1 Key Regimes in the WFO Window (2022–2025)

The four OOS windows span two distinct macroeconomic regimes. This is a short window by institutional standards—one of the known limitations of a yfinance-only build. It is still sufficient to validate the M/G/$\infty$ allocation math.

| **Period** | **Regime** | **Expected Behaviour** | **M/G/$\infty$ Stress** |
|---|---|---|---|
| 2022 | Inflation + aggressive rate hikes | Growth stock collapse. Value signals fire heavily in Tech and Consumer. $\lambda$ elevated. | Tests if $k$ buffer is calibrated correctly when trained on 2020–2021. |
| 2023 | Soft landing + AI boom | Rapid Tech rerating. Short holding durations. High $\mu$, low $\mu^{-1}$. | Low stockout pressure. Tests if $R_i$ shrinks appropriately. |
| 2024–2025 | Rate cuts + broad recovery | Moderate $\lambda$ across sectors. Strategy expected to perform steadily. | Baseline regime—results here most representative. |

## 7.2 2020 Covid Crash: A Training Data Asset

In this v1 build, 2020 falls inside the training window, not the OOS period. This is actually useful: the Covid crash is the most extreme $\lambda$ spike in recent history—every stock looked cheap simultaneously by P/E standards. Training on this event means the calibrated $k$ buffer will incorporate the experience of a systemic shock. The model will set a higher $k$ than if trained only on calm 2023–2024 data.

The diagnostic question for the 2022 OOS window is: *did the model trained on 2020–2021 (including the crash and V-recovery) produce a $k$ buffer large enough to handle the more moderate but sustained $\lambda$ elevation of the inflation period?* If the stockout rate in 2022 is near the 1% target, the training data inclusion was beneficial. If it is far above 1%, the 2020 spike overfit $k$ in the wrong direction for a sustained-inflation regime.

# Part IV

# Appendices

# Chapter 8

# Database Setup and Operations

## 8.1 PostgreSQL Installation and Configuration

```python
# Install PostgreSQL and Python dependencies
# Run in terminal:
# sudo apt-get install postgresql postgresql-contrib  (Linux)
# brew install postgresql  (macOS)

# Python dependencies
# pip install psycopg2-binary pandas numpy scipy yfinance requests

# Create the database
# sudo -u postgres psql
# CREATE DATABASE quant_db;
# CREATE USER quant_user WITH ENCRYPTED PASSWORD 'your_password';
# GRANT ALL PRIVILEGES ON DATABASE quant_db TO quant_user;

DB_CONFIG = {
    "dbname":   "quant_db",
    "user":     "quant_user",
    "password": "your_password",
    "host":     "localhost",
    "port":     5432
}

import psycopg2

def initialize_database():
    """Create all tables and indexes from scratch."""
    conn = psycopg2.connect(**DB_CONFIG)
    cur  = conn.cursor()

    # Read the SQL schema files and execute
    schema_files = [
        'sql/01_tickers_metadata.sql',
        'sql/02_daily_prices.sql',
        'sql/03_fundamentals.sql',
    ]
    for f in schema_files:
        with open(f) as fh:
            cur.execute(fh.read())
        conn.commit()
        print(f"Executed: {f}")

    cur.close()
    conn.close()
```

```
    print("Database initialized successfully.")
```

Listing 8.1: Environment Setup Script

## 8.2 Master Orchestration Script

```
import logging
import pandas as pd
from datetime import datetime

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[
        logging.FileHandler('quant_pipeline.log'),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger(__name__)

def run_full_pipeline():
    """
    End-to-end data pipeline.
    Estimated total runtime on first run: ~2-3 hours.
      Phase 1: < 1 minute    (CSV load)
      Phase 2: 30-90 min     (price download, batched)
      Phase 3: 45-90 min     (fundamentals, one-by-one)
      Phase 4: < 5 min       (WFO on 4 windows)
    """
    start_time = datetime.now()
    logger.info("=== QUANT PIPELINE STARTED ===")

    # PHASE 1: Universe -- straight from IWV_holdings25.csv
    logger.info("Phase 1: Loading fixed universe from IWV_holdings25.csv")
    universe = load_iwv_universe("IWV_holdings25.csv")
    insert_universe_to_db(universe)
    tickers = universe["ticker"].tolist()
    logger.info(f"Phase 1 complete: {len(tickers):,} tickers loaded")

    # PHASE 2: Price data (2010-present) via yfinance, batched by 100
    logger.info("Phase 2: Downloading price data via yfinance (2010-present)")
    bulk_download_prices(tickers, start="2010-01-01", batch_size=100)
    logger.info("Phase 2 complete")

    # PHASE 3: Quarterly fundamentals via yfinance (one ticker at a time)
    logger.info("Phase 3: Downloading quarterly fundamentals via yfinance")
    logger.info("  NOTE: yfinance covers ~2019-present. Pre-2019 will be empty.")
    load_all_fundamentals(tickers)
    logger.info("Phase 3 complete")

    # PHASE 4: Walk-Forward Optimization (2020-2025, 2yr train, 1yr OOS)
    logger.info("Phase 4: Running Walk-Forward Optimization (2022-2025 OOS)")
    results = run_walk_forward_optimization(
        total_capital_usd = 500_000_000,
        train_years       = 2,
        oos_years         = 1,
        wfo_start         = "2020-01-01",
        wfo_end           = "2025-12-31",
        target_alpha      = 0.01
```

```
    )
    results.to_csv('wfo_results.csv', index=False)
    logger.info(f"Phase 4 complete. {len(results)} OOS windows. Saved to
    wfo_results.csv")

    elapsed = datetime.now() - start_time
    logger.info(f"=== PIPELINE COMPLETE in {elapsed} ===")

if __name__ == "__main__":
    run_full_pipeline()
```

Listing 8.2: Main Pipeline Runner

# Chapter 9

# Data Quality Checklist

Before running any WFO, verify your database passes these checks. All queries use the three actual table names: `tickers_metadata`, `daily_prices`, `fundamentals`.

**Check 1. Universe Completeness.** Confirm you loaded all 2,645 expected equity tickers from the IWV CSV.

```sql
SELECT COUNT(*) AS n_tickers FROM tickers_metadata;
-- Expected: 2645
-- If lower: re-run load_iwv_universe() and check the Asset Class filter
```

**Check 2. Sector Coverage.** Verify all 11 GICS sectors are present and counts look reasonable.

```sql
SELECT sector, COUNT(*) AS n
FROM tickers_metadata
GROUP BY sector ORDER BY n DESC;
-- Expected top sectors: Financials ~476, Health Care ~456, Industrials ~399
-- If any sector is NULL: some tickers had blank Sector in the CSV
```

**Check 3. Price Coverage.** Check that the bulk download actually ran. Should have millions of rows.

```sql
SELECT COUNT(*) AS total_rows,
       COUNT(DISTINCT ticker) AS tickers_with_prices,
       MIN(date) AS earliest, MAX(date) AS latest
FROM daily_prices;
-- Expected: >5M rows, ~2600+ tickers, earliest ~2010-01-04
```

**Check 4. Price Gap Check.** Flag tickers with more than 10 consecutive missing trading days.

```sql
SELECT ticker,
       MAX(date - LAG(date) OVER (PARTITION BY ticker ORDER BY date))
           AS max_gap_days
FROM daily_prices
GROUP BY ticker
HAVING MAX(date - LAG(date) OVER (PARTITION BY ticker ORDER BY date)) > 10
ORDER BY max_gap_days DESC
LIMIT 20;
```

```
-- Any result > 30 days is suspicious (IPO gap is expected, not mid-history)
```

**Check 5. Fundamentals Coverage.** Verify fundamentals loaded and cover the expected date range.

```
SELECT COUNT(*) AS total_rows,
       COUNT(DISTINCT ticker) AS tickers_with_fundamentals,
       MIN(period_end_date) AS earliest, MAX(period_end_date) AS latest
FROM fundamentals
WHERE eps_diluted IS NOT NULL;
-- Expected: earliest ~2019-Q1 (yfinance limitation)
-- If tickers_with_fundamentals << 2000: yfinance rate-limited, re-run
```

**Check 6. Signal Sanity.** Run the signal generator on one recent date and confirm it produces output.

```
-- In Python: signals = generate_signals('2024-06-30')
-- Expected: 50-300 signals depending on market conditions
-- If 0 signals: fundamentals table is empty or period_end_date filter too strict
-- If > 1000 signals: check pe_ratio outlier filter (pe > 1 and pe < 200)
```

# Chapter 10

# Glossary of Key Terms

**M/G/$\infty$ Queue**
: A queuing model where arrivals are Markovian (Poisson), service times follow a General distribution, and there are infinitely many servers. The number of active jobs at any time follows a Poisson distribution with mean $R = \lambda/\mu$.

**Square-Root Rule**
: The optimal capital allocation formula $c_i = R_i + k\sqrt{R_i}$ derived from the M/G/$\infty$ model, ensuring equal stockout probability across all sectors.

**Survivorship Bias**
: The statistical error of only analyzing entities that survived to the present, ignoring those that failed. In finance: a backtest that excludes bankrupt companies will show artificially inflated returns. *This implementation intentionally accepts this bias as a simplification. The fixed-universe approach means all 2025 Russell 3000 companies "survived"—the bankruptcies, delistings, and value traps of 2010–2024 are excluded. Future versions should correct this using historical 13F-based universe construction.*

**Point-in-Time (PiT)**
: A database design principle where every data point is tagged with the date it became publicly available, preventing any future information from contaminating historical simulations.

**Look-Ahead Bias**
: The error of using information in a backtest that would not have been available at the simulated trading date. The primary cause of falsely profitable backtest results.

**Walk-Forward Optimization (WFO)**
: A validation methodology that slides a training window forward through time, recalibrating model parameters annually and testing on unseen data, replicating the experience of a live portfolio manager.

**Offered Load ($R_i$)**
: In M/G/$\infty$ queuing theory, the expected number of concurrent jobs in sector $i$: $R_i = \lambda_i \cdot E[S_i]$. In portfolio terms: the expected number of simultaneously open positions in a sector.

**Stockout Risk** The probability that a new buy signal arrives when no capital is available in the relevant sector. The M/G/∞ framework minimizes this while constraining total capital.

**XBRL** eXtensible Business Reporting Language. The machine-readable format mandated by the SEC for financial filings since 2009. The basis for free, clean fundamental data extraction via EDGAR API.

**13F Filing** A quarterly SEC filing required of all institutional managers with over $100M in assets, disclosing all equity holdings. The iShares IWV ETF's 13F filings can be used to reconstruct historical Russell 3000 membership. *Not used in this simplified version, which sources the universe from the current IWV holdings CSV instead.*

**CUSIP** Committee on Uniform Securities Identification Procedures. A permanent 9-digit identifier for securities. Unlike tickers, CUSIPs do not change when a company renames or re-lists.

# Final Note

Building this infrastructure is a tractable project. The hardest part—universe construction—is already done: you have `IWV_holdings25.csv`. Phase 1 is a one-minute DB insert. Phase 2 (prices) and Phase 3 (fundamentals) are both yfinance downloads that run unattended in 2–3 hours total on a standard laptop. Phase 4 (WFO) runs in minutes once the data is loaded.

The right order is strictly sequential: do not run the WFO until the data quality checklist passes. Specifically, do not run signal generation until you have confirmed that `fundamentals` has rows with valid `eps_diluted` values for recent quarters.

**The one thing most likely to go wrong.** yfinance is an unofficial API with no SLA. It rate-limits aggressively and silently returns empty DataFrames for some tickers. The fundamentals loop will produce gaps. The fix: run it twice—on the second pass, only fetch tickers that have no rows in `fundamentals`—a trivial 10-line modification. Build this retry logic before you run Phase 3 the first time.

**The M/G/$\infty$ edge.** The strategy's advantage is not the valuation signal—mean-reversion on P/E is a well-known factor. The edge is the *allocation discipline*. By treating capital deployment as a queuing problem with mathematically derived sector budgets, the strategy avoids the two most common discretionary errors: over-concentrating in a sector when signals cluster (leading to correlated drawdowns) and under-deploying capital in sectors with slow, high-value signals. The Square-Root Rule enforces a position-sizing regime that is both theoretically grounded and practically robust across market regimes. The 4-year OOS window this infrastructure produces is enough to see whether that discipline actually reduces stockout rates versus a naive equal-weight sector allocation—which is the core question this prototype is designed to answer.

**Future Extensions (in priority order).** (1) **Deeper fundamentals**: replace yfinance with SEC EDGAR XBRL for pre-2019 coverage and true PiT filing dates—extends the OOS window from 4 to 10 years. (2) **Survivorship bias correction**: replace fixed `tickers_metadata` with a time-series table built from IWV 13F filings. (3) **Signal enrichment**: add P/B and EV/EBITDA alongside P/E for a composite valuation z-score. None of these require changes to the WFO engine or the M/G/$\infty$ allocation math—they are clean data layer upgrades.

**The M/G/∞ edge.** The strategy's advantage is not the valuation signal—mean-reversion on P/E is a well-known factor. The edge is the *allocation discipline*. By treating capital deployment as a queuing problem with mathematically derived sector budgets, the strategy avoids the two most common discretionary errors: over-concentrating in a sector when signals cluster (leading to correlated drawdowns) and under-deploying capital in sectors with slow, high-value signals. The Square-Root Rule enforces a position-sizing regime that is both theoretically grounded and practically robust across market regimes.